

Product Requirements Document

LLM Powered AI Chatbot

Version 1.0 | Development Ready

1. Product Overview

The product is a web based AI chatbot that allows users to interact with Google's Gemini large language model through a modern conversational interface.

The application will provide real time AI responses, conversation history, Markdown rendering, code formatting, error handling, and persistent conversations.

Technology stack: React, Django, Django REST Framework, PostgreSQL, and Google Gemini API.

2. Product Goal

Build a reliable, responsive, and extensible AI chatbot that demonstrates the complete lifecycle of an LLM powered application.

Users should be able to start conversations, send messages to Gemini, receive streaming responses, continue conversations with context, view previous conversations, render Markdown and code, handle errors, and securely persist their conversations.

3. Target Users

Primary users are developers and technical learners who want to interact with an AI assistant.

The architecture should be extensible to support students, professionals, teams, and businesses.

4. Core User Journey

User opens the application → logs in → starts a conversation → enters a message → React sends the request to Django → Django validates it and loads context → Django sends the request to Gemini → Gemini streams the response → Django streams it to React → React displays it → the conversation is persisted.

5. Functional Requirements

5.1 Authentication

- Registration
- Login
- Logout
- Authentication persistence
- Protected API endpoints
- User specific conversations

5.2 Chat Interface

- New conversation
- Message input

- Send message
- Immediate display of user messages
- Streaming AI responses
- Conversation switching
- Conversation sidebar

5.3 Gemini Integration

Django must communicate with Gemini. The Gemini API key must remain on the Django server. Gemini logic should be isolated in a service layer.

5.4 Prompt Engineering

Support configurable system instructions and construct model context from system instructions, conversation history, and the current user message.

5.5 Conversation History

Previous messages must be loaded and supplied to Gemini so the model can maintain conversational context.

5.6 Conversation Management

Users can create, view, rename, switch, and delete conversations.

5.7 Message Management

Messages should store conversation, role, content, and timestamps. Supported roles include user, assistant, and system.

6. Streaming AI Responses

AI responses must stream progressively so users see content as it is generated rather than waiting for the complete response.

The intended flow is React → Django → Gemini → Django streaming response → React.

7. Markdown and Code Rendering

The frontend should render headings, bold text, italic text, lists, links, blockquotes, tables, inline code, and fenced code blocks.

Code blocks should support syntax highlighting, language identification, horizontal scrolling, and copy functionality. Model generated content must be handled safely and must not be blindly injected as arbitrary HTML.

8. Loading and Error States

The interface should distinguish idle, sending, streaming, completed, and error states.

The system must handle invalid requests, authentication failures, Gemini failures, rate limits, network failures, timeouts, database failures, invalid conversations, and context limits. Technical details should be logged server side while users receive clear messages.

9. PostgreSQL Requirements

PostgreSQL will persist users, conversations, and messages.

Initial entities:

User

Conversation: id, user, title, created_at, updated_at

Message: id, conversation, role, content, created_at

The architecture should allow future entities such as usage records, prompt templates, API logs, subscriptions, documents, and embeddings.

10. API Requirements

Authentication:

POST /api/auth/register/

POST /api/auth/login/

POST /api/auth/logout/

Conversations:

GET /api/conversations/

POST /api/conversations/

GET /api/conversations/{id}/

PATCH /api/conversations/{id}/

DELETE /api/conversations/{id}/

Chat:

POST /api/chat/

Example request:

```
{  
  "conversation_id": 12,  
  "message": "Explain React hooks"  
}
```

The chat endpoint must support streaming.

11. Security Requirements

The Gemini API key must never be exposed to React.

Required controls:

- Authentication
- Authorization
- Input validation
- Rate limiting
- CSRF protection where applicable
- Correct CORS configuration

- Secure environment variables
- SQL injection protection
- XSS protection
- Request logging

Users must only be able to access their own conversations.

12. Rate Limiting and Usage Tracking

The backend should enforce server side rate limits to protect the Gemini API from abuse.

The system should be designed to track user, conversation, model, request count, input tokens, output tokens, response time, and errors.

13. Frontend Architecture

Recommended structure:

src/

components/

ChatWindow.jsx

Message.jsx

ChatInput.jsx

Sidebar.jsx

ConversationList.jsx

LoadingIndicator.jsx

CodeBlock.jsx

pages/

Login.jsx

Register.jsx

Chat.jsx

api/

auth.js

chat.js

conversations.js

hooks/

useChat.js

App.jsx

main.jsx

14. Django Architecture

Recommended structure:

backend/

config/

users/

conversations/

chat/

models.py

serializers.py

views.py

urls.py

services/

gemini.py

prompts.py

streaming.py

requirements.txt

manage.py

Business logic should be placed in service layers rather than concentrated inside views.

15. Non Functional Requirements

Performance: display user messages immediately, begin streamed responses quickly, avoid unnecessary database queries, and keep the interface responsive.

Reliability: gracefully handle LLM, network, database, and invalid request failures.

Scalability: allow future support for more users, conversations, LLM providers, RAG, file uploads, AI agents, subscriptions, and teams.

16. MVP Scope

Authentication: registration, login, logout.

Chat: new conversation, send message, Gemini response, streaming, conversation context.

Persistence: PostgreSQL, conversation storage, message storage.

UI: chat interface, conversation sidebar, loading states, error states, Markdown, code blocks, copy code.

Security: server side Gemini API key, authentication, authorization, rate limiting, and input validation.

17. Future Features

Potential future capabilities include file uploads, PDF conversations, RAG, vector search, web search, voice input, voice output, image understanding, image generation, multiple Gemini models, model selection, conversation

sharing, export, team workspaces, subscriptions, usage dashboards, admin dashboards, AI agents, and tool calling.

18. Development Milestones

1. Project setup
2. Django REST API
3. PostgreSQL integration
4. Gemini API integration
5. Basic React chat
6. Conversation context
7. Streaming
8. Markdown and code rendering
9. Authentication
10. Conversation persistence
11. Error handling and rate limiting
12. Production deployment

19. Definition of Done

The MVP is complete when an authenticated user can register, log in, create conversations, send messages, receive Gemini responses, see responses stream, continue conversations, return later and see previous conversations, open old conversations, continue chatting, view Markdown and syntax highlighted code, copy code, receive useful errors, and access only their own data.

20. Final Technology Stack

Frontend: React + Tailwind CSS

Backend: Django + Django REST Framework

Database: PostgreSQL

AI: Google Gemini API

Authentication: Django authentication / JWT

Markdown: React Markdown

Code highlighting: Syntax highlighting library

Version control: Git + GitHub

Deployment: React frontend + Django backend + PostgreSQL

21. Final Architecture

User

↓

React

↓ HTTPS

Django

■■■ PostgreSQL

■■■ Gemini API

React owns the user experience. Django owns application and AI orchestration logic. PostgreSQL owns persistent state. Gemini owns language generation.